

IncidentIQ - Post-Incident Report (PIR)

Generated via AI Multi-Agent Audit Pipeline

Execution Metadata & Configuration	
Model: gemini-3.5-flash	Mode: Standard Analysis
Audience: Support Team	Analysis Response Length: Normal

Formal Postmortem

1. Executive Summary & Impact

On July 27, 2026, at 14:15:00 UTC, a scheduled rolling update to `Auth Backend v3.0` was initiated. Following the completion of the deployment at 14:17:30, compatibility issues immediately arose. While newer clients (such as Android App v4.2) authenticated successfully, legacy clients running older application versions (specifically iOS App v3.1) experienced immediate authentication failures.

The validation logic in `Auth Backend v3.0` strictly required a new JWT claim, `device\_hash`, which legacy iOS clients do not generate. Consequently, these clients received `HTTP 401 Unauthorized` errors. Automated monitoring detected a 100% login failure rate on the iOS client cluster at 14:21:00. To restore service, the system successfully initiated a rollback to `v2.9` at 14:23:45.

Impact Metrics

- * Total Downtime / Degradation: 6 minutes and 15 seconds (from the first failure at 14:19:05 to the rollback initiation at 14:23:45).
- * Targeted Users: 100% of users attempting to authenticate via legacy iOS App v3.1.
- * Total Time to Detect (TTD): 1 minute, 55 seconds (from first error to automated alert).
- * Total Time to Mitigate (TTM): 4 minutes, 40 seconds (from first error to rollback initiation).
- * Service Level Indicator (SLI): Authentication success rate dropped to 0% for the legacy iOS cluster during the incident window.

2. Incident Timeline

Timestamp (UTC)	Severity	Source	Description
:--	:--	:--	:--
14:15:00	INFO	AuthService	Rolling update to Auth Backend v3.0 initiated in production.
14:17:30	INFO	AuthService	Update completes successfully; the backend begins handling live incoming traffic.
14:18:10	INFO	LoginAPI	Successful authentication of User 992 running Android App v4.2 (modern client).
14:19:05	ERROR	LoginAPI	First Failure: JWT validation fails for User 1045 running legacy iOS App v3.1 due to missing `device_hash` claim.
14:19:10	ERROR	Frontend	User receives an `HTTP 401 Unauthorized` response.
14:20:15	ERROR	LoginAPI	Second validation failure logged for User 2088 (iOS App v3.1), confirming a systemic pattern.
14:21:00	WARN	Monitor	Automated monitoring triggers an alert: 100% login failure rate detected on the iOS client cluster.

IncidentIQ - Post-Incident Report (PIR)

Generated via AI Multi-Agent Audit Pipeline

| 14:23:45 | `FATAL` | `System` | System marks the auth pipeline as degraded for legacy clients and automatically triggers a rollback to v2.9. |

3. Root Cause Analysis (5 Whys)

- To understand the systemic weaknesses that allowed this failure to reach production, we perform a 5 Whys analysis:
- Why did legacy iOS users experience 100% login failures?
 - * They received `HTTP 401 Unauthorized` errors because the updated `LoginAPI` rejected their authentication requests.
 - Why did the LoginAPI reject these requests?
 - * The JWT validation logic failed due to a missing required `device\_hash` claim in the client tokens.
 - Why was the `device\_hash` claim missing from the legacy iOS clients' tokens?
 - * The legacy iOS App v3.1 client was designed and released before the `device\_hash` schema was introduced, and it does not possess the code required to generate or pass this claim.
 - Why did the new Auth Backend v3.0 enforce this claim as strictly mandatory for all active clients without backward compatibility?
 - * The backend's schema validation was configured to strictly reject any token missing this claim on all endpoints, lacking a feature flag or fallback mechanism to support older, still-supported client versions.
 - Why was this compatibility gap between Auth Backend v3.0 and legacy clients not caught during testing or staging?
 - * Our pre-deployment integration test suites only validated "happy path" scenarios using mock payloads representing the latest client versions (e.g., Android v4.2). There was no test coverage mapping deprecated or legacy client configurations against the new JWT validation schema.

4. Lessons Learned & Action Items

Lessons Learned

- * Testing Gaps: Testing must always encompass backward-compatibility profiles. Validating only the latest client releases exposes the system to breaking changes for legacy users.
- * Graceful Degradation / Feature Flagging: Hard-enforcing new schema requirements without a transitional phase or grace period presents an immediate risk to service reliability. New claims should initially be optional, warnings logged, and then strictly enforced only after client adoption meets a threshold.
- * Rapid Mitigation: The automated alerting and quick rollback initiation minimized the blast radius of this incident, validating our monitoring thresholds for cluster-specific anomalies.

Action Items

Action Item	Type	Priority	Target Date	Owner
Modify JWT Validation				
Update the `device_hash` validation logic to be optional/warn-only for client versions below				

IncidentIQ - Post-Incident Report (PIR)

Generated via AI Multi-Agent Audit Pipeline

iOS v4.0 to allow backward compatibility. | Corrective | Critical | 2026-07-29 | Auth Team |

| Expand CI/CD Integration Test Suites
Add backward-compatibility integration tests to the deployment pipeline that simulate payloads from all active legacy client versions (e.g., iOS v3.1). | Preventative | High | 2026-08-05 | QA / DevOps |

| Implement Canary Deployments
Transition `AuthService` deployments from rolling updates to a multi-phase Canary model (e.g., 1% -> 10% -> 100%) to isolate and capture client-specific failures before full rollouts. | Preventative | High | 2026-08-12 | SRE |

| Establish an API/JWT Schema Deprecation Policy
Formulate a structured deprecation and migration checklist for any future changes to authentication tokens, ensuring clear timelines for sunseting legacy apps before enforcing backend changes. | Process | Medium | 2026-08-19 | Product / Security |

AI Investigation Report

Hello Team,

As Lead SRE, I have investigated the authentication issues reported this afternoon. Below is a structured analysis of the incident, what we know, and the steps we are taking to resolve it.

Please use this guide to assist users who write in with login issues.

1. Timeline of events

- * 14:15:00 - The SRE team initiated a rolling update to deploy Auth Backend v3.0.
- * 14:17:30 - The update completed successfully, and v3.0 began handling live user traffic.
- * 14:18:10 - Successful login observed for an Android user (User 992 on Android App v4.2), indicating the new backend is functional for some clients.
- * 14:19:05 - First error observed: iOS User 1045 (using iOS App v3.1) failed validation because their login token (JWT) was missing a required `device\_hash` claim.
- * 14:19:10 - The system blocked the request, returning an HTTP 401 Unauthorized error to client IP `192.168.1.55`.
- * 14:20:15 - Second error observed: Another iOS user (User 2088 on iOS App v3.1) failed validation for the exact same reason (missing `device\_hash`).
- * 14:21:00 - Automated monitoring systems flagged a 100% login failure rate for the entire iOS client cluster.
- * 14:23:45 - System classified the pipeline as degraded for legacy clients and automatically initiated a rollback to Auth Backend v2.9.

2. Facts vs. Assumptions

- * Facts:
- * The deployment of Auth Backend v3.0 directly triggered the login failures.

IncidentIQ - Post-Incident Report (PIR)

Generated via AI Multi-Agent Audit Pipeline

- * Android App v4.2 users can log in successfully on v3.0.
- * iOS App v3.1 users are completely blocked from logging in on v3.0.
- * The specific technical error is a missing `device\_hash` parameter in the security token sent by iOS v3.1.
- * The system has initiated an automatic rollback to the previous working version (v2.9).
- * Assumptions:
 - * *Assumption:* iOS App v3.1 is considered a "legacy client," whereas Android App v4.2 is a modern client that already generates and sends the `device\_hash`.
 - * *Assumption:* The rollback to v2.9 will fully restore login capabilities for iOS v3.1 users without needing an app update.
 - * *Assumption:* There might be a newer iOS app version (e.g., v4.x) that does not experience this issue.

3. Evidence-For vs. Evidence-Against

Hypothesis	Evidence-For	Evidence-Against
Backend v3.0 introduced a breaking change for older clients.	* Log shows validation failed immediately after the v3.0 update.	
* Specific error is a "missing required claim," which is typically enforced by backend security updates. | * Android App v4.2 logged in successfully, showing the backend is not completely broken for all users. |
| The iOS App v3.1 client has a bug/outage independent of the backend. | * Only iOS App v3.1 clients are failing validation. | * These clients worked fine before 14:17:30. The issues started exactly 95 seconds after the v3.0 backend went live. |
| There is a wider network or database outage. | * 100% failure rate flagged on iOS cluster. | * Android user authenticated successfully at 14:18:10; the errors are highly specific to missing token claims. |

4. Top Root-Cause Hypotheses

Hypothesis 1: Backward Compatibility Failure in Auth Backend v3.0 (High Confidence)

- * Description: Auth Backend v3.0 made the `device\_hash` security claim mandatory for all incoming login requests. However, legacy app versions (specifically iOS App v3.1) were never programmed to generate or send this claim. Because the backend is strictly enforcing this new rule, it is rejecting all legacy iOS logins.
- * Impact: 100% of users on iOS App v3.1 (and potentially other legacy versions) cannot log in.

Hypothesis 2: Misconfigured Feature Flag in Production (Medium Confidence)

- * Description: The `device\_hash` requirement was meant to be turned off or optional for older clients via a configuration setting/feature flag, but it was deployed to production as "strictly enforced" by mistake.

5. Recommended Debugging Actions

IncidentIQ - Post-Incident Report (PIR)

Generated via AI Multi-Agent Audit Pipeline

If customers contact Support regarding login failures, please follow these steps:

1. Verify the Rollback Status (Tier-1/Tier-2):

- * Check our internal system status dashboard to confirm that the rollback to Auth Backend v2.9 has successfully completed. Once the rollback is complete, iOS v3.1 users should immediately be able to log in again.

2. Collect Client Information from Affected Users:

- * If a user reports a login failure, ask them for:
 - * Operating System (iOS or Android).
 - * App Version (e.g., v3.1).

3. Recommend the Temporary Workaround:

- * Ask the user to check the App Store to see if there is an update available for our app (e.g., v4.0 or higher). Upgrading to the latest app version will likely resolve the issue.

4. How to Escalate:

- * If a customer reports a 401 Unauthorized/Login error *after* SRE has confirmed the rollback to v2.9 is complete, escalate the ticket to the SRE-On-Call channel on Slack immediately with the user's ID and App Version.

Skeptical Audit & Biases

1. Identify unsupported claims, ungrounded speculation, or hallucinations in the initial analysis.

- * Existence of iOS App v4.0+: The initial analysis recommends that support agents instruct users to upgrade to "v4.0 or higher" to resolve the issue. There is absolutely no mention of any iOS version other than v3.1 in the provided logs. The existence of a newer iOS version is a hallucination.

- * Widespread Android Success: The initial analysis lists as a "Fact" that "Android App v4.2 users can log in successfully." The logs contain only a single log entry of a successful login for one Android user (User 992). Extrapolating a single successful event to declare that the entire client version is fully functional is an unsupported claim.

- * Feature Flag Misconfiguration (Hypothesis 2): The speculation regarding a "misconfigured feature flag" has no supporting evidence in the logs. The logs do not reference configuration files, feature flags, or deployment settings.

- * iOS v3.1 as a "Legacy Client": While the system log at 14:23:45 notes that the "pipeline [is] degraded for legacy clients," the initial analysis categorically labels iOS v3.1 as a legacy client in its facts and assumptions. The logs do not define which app versions are officially categorized as "legacy."

2. Highlight any Post Hoc Fallacy (e.g., assuming a deployment caused the failure solely because it followed).

- * Assumption of Direct Causation: The initial analysis asserts as a hard fact that "The deployment of Auth Backend v3.0 directly triggered the login failures." This is a classic **post hoc ergo propter hoc** (after this, therefore because of this) fallacy.

- * Temporal Correlation vs. Causation: The conclusion is based entirely on the timeline: the deployment completed at 14:17:30, and the first error occurred at 14:19:05. While temporal correlation is a strong indicator in incident response, the logs do not provide diagnostic proof that the backend update itself introduced the validation requirement. It is theoretically possible that an external token-issuing authority (such as an identity provider) altered its JWT payload

IncidentIQ - Post-Incident Report (PIR)

Generated via AI Multi-Agent Audit Pipeline

format independent of, but concurrent with, the deployment.

3. Point out any Confirmation Bias or Automation Bias in the reasoning.

- * Confirmation Bias: The initial analysis exhibits confirmation bias by focusing on the single successful Android login at 14:18:10 to validate the hypothesis that the backend is functional for non-legacy clients. It ignores alternative possibilities-such as the Android client using a cached token, bypassing certain validation pathways, or being routed to a different, non-upgraded node.

- * Automation Bias (Monitoring & Rollback): The initial analysis exhibits automation bias by blindly trusting the automated systems. It accepts the automated monitoring system's "100% login failure rate" flag for iOS and the system's "rollback" log as definitive proof that the issue is resolved. The analysis instructs support staff to assume the rollback "will fully restore login capabilities... immediately" without recommending manual verification of the success of the rollback or monitoring post-rollback telemetry.

4. What critical questions did the initial analysis fail to ask?

- * Was iOS App v3.1 sending the ``device\_hash`` claim prior to the v3.0 deployment? The analysis assumes the client never sent it, but it is critical to verify if a change in the client or a third-party token generator occurred.

- * Are there other active iOS app versions? The logs only show failures for iOS v3.1. It is critical to know if other iOS versions are logging in successfully, which would help isolate whether the issue is OS-specific or version-specific.

- * What component actually generates the JWT? The logs show "JWT Validation failed," which occurs on the backend, but the token is generated elsewhere (likely a client-side library or an authentication provider). The analysis fails to ask if the token generator itself experienced an outage or update.

- * Why is there only one successful Android login? If the backend were fully functional for Android, a higher volume of successful logins would be expected between 14:17:30 and 14:23:45. The lack of other successful login logs should be investigated.